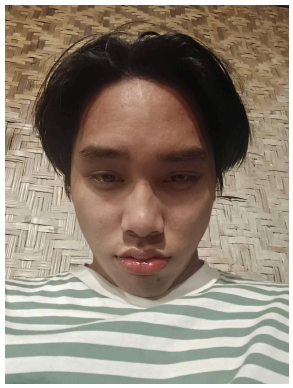
TUGAS BESAR

IF2224 - Teori Bahasa Formal dan Otomata

Milestone 1

LEXICAL ANALYSIS



Benedictus Nelson - 13523150

Rainaldi Pratama F. Sembiring - 13524117

Jonathan Alveraldo Bangun - 13524120

Ramadhian Nabil Firdaus Gumay - 13524126

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2026

A. TEORI SINGKAT

Data Flow Diagram adalah representasi grafik yang menggambarkan aliran data melalui sebuah sistem informasi. DFD membantu analis sistem dan pengembang untuk memahami bagaimana data masuk, diproses, disimpan, dan dikeluarkan dari sistem tanpa terikat pada implementasi teknis (seperti bahasa pemrograman atau perangkat keras).

Komponen Utama:

a. Entitas Luar (External Entity): Sumber atau tujuan data di luar sistem.

b. Proses (Process): Transformasi input menjadi output.

c. Simpanan Data (Data Store): Tempat penyimpanan data (seperti basis data atau file).

d. Arus Data (Data Flow): Jalur perpindahan data antar komponen.

Lexer (sering disebut juga sebagai *Scanner*) adalah fase pertama dalam proses kompilasi sebuah kode program. Tugas utamanya adalah membaca urutan karakter dari kode sumber (source code) dan mengelompokkannya menjadi unit-unit bermakna yang disebut Lexemes.

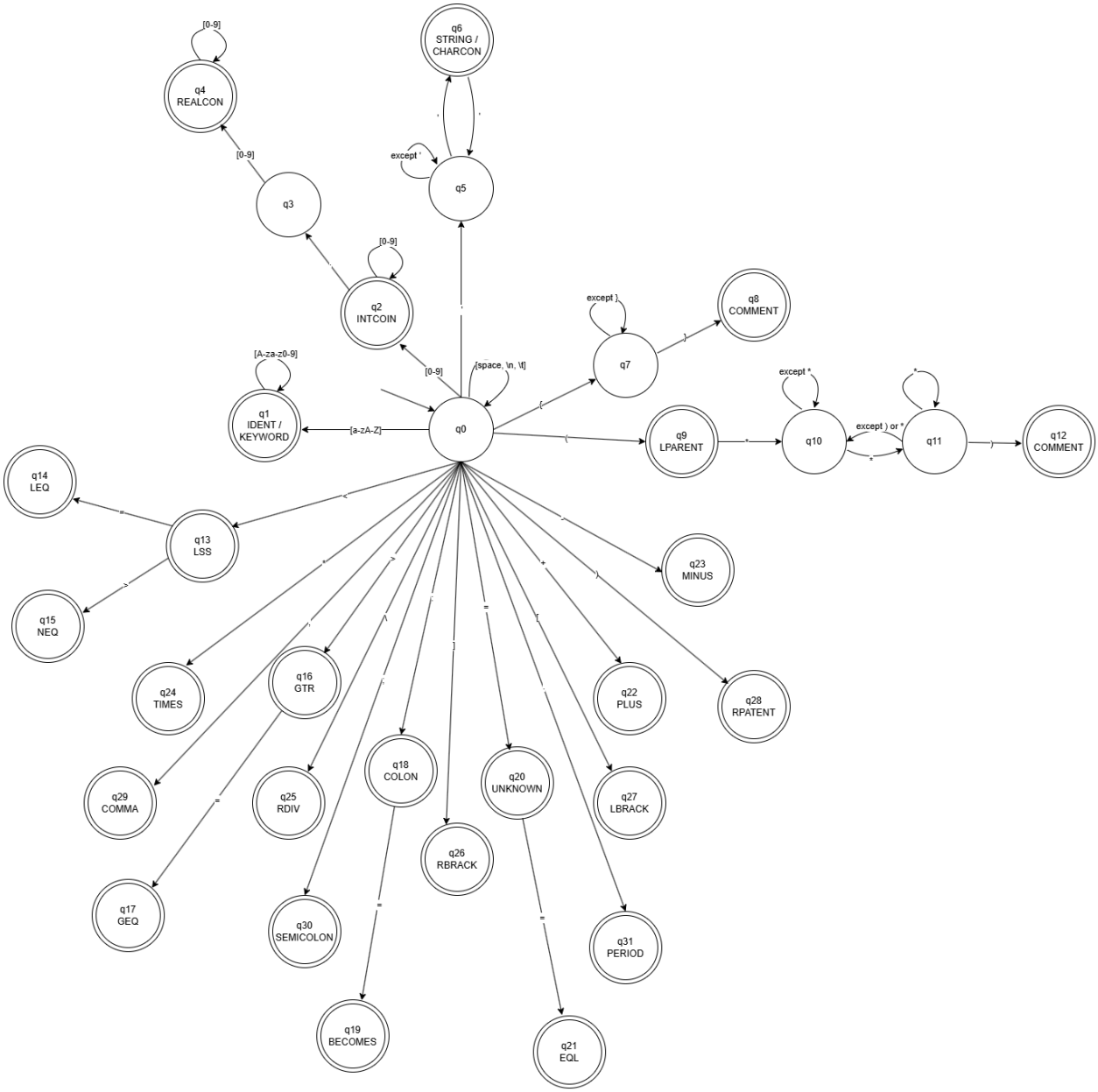
Lexer menyisir kode baris demi baris, mengabaikan spasi kosong (*whitespace*) atau komentar, dan mengidentifikasi pola tertentu menggunakan aturan Regular Expressions.

Token adalah representasi simbolik atau kategori dari sebuah *lexeme* yang dihasilkan oleh Lexer. Token inilah yang nantinya dikirim ke fase berikutnya (Parser) untuk diperiksa struktur sintaksisnya. Setiap Token biasanya terdiri dari dua bagian: Nama Token (kategori) dan Nilai Atribut (isi spesifiknya).

Meskipun ketiganya berada dalam ranah ilmu komputer, DFD digunakan pada tahap Analisis dan Perancangan Sistem, sedangkan Lexer dan Token bekerja pada tahap Implementasi/Eksekusi (khususnya dalam pembuatan *interpreter* atau kompilator bahasa pemrograman).

B. PERANCANGAN DAN IMPLEMENTASI

1. Perancangan Diagram Transisi DFA



a. Ketika berada di state awal q0, karakter whitespace (spasi, tab, newline) tidak akan memicu perpindahan state atau pembentukan token. Mesin hanya akan melakukan looping pada q0 dan mengabaikan karakter tersebut untuk melanjutkan ke karakter berikutnya.

b. Transisi ke q1 terjadi jika karakter pertama yang dibaca adalah huruf. Di q1, mesin akan terus menerima kombinasi huruf maupun angka. State q1 ini merupakan final state yang digeneralisasi untuk token IDENT dan semua jenis keyword (seperti if, while, begin). Penentuan apakah suatu kata merupakan identifier atau keyword dilakukan lebih lanjut pada tahap pengecekan kode (checkKeyword), sesuai dengan izin penyederhanaan state pada spesifikasi.

c. Pembacaan angka diawali dengan transisi ke q2 yang merupakan final state untuk token INTCON. Jika setelah angka mesin membaca karakter titik (.), mesin akan transit sementara ke q3 untuk melakukan validasi lanjutan (peek). Jika karakter berikutnya adalah angka, mesin bertransisi ke q4 dan menjadikannya token desimal REALCON. Namun, jika karakter berikutnya setelah titik adalah titik lagi (merepresentasikan range array seperti 1..10), mesin tidak akan masuk ke REALCON, melainkan menyelesaikan token pada titik pertama tersebut sebagai INTCON.

d. Karakter kutip tunggal (') pembuka akan membawa mesin dari q0 ke q5 untuk mulai membaca isi teks. Ketika bertemu kutip tunggal penutup, mesin masuk ke q6 yang merupakan final state untuk STRING atau CHARCON. Diagram ini juga mengakomodasi penanganan escape character, di mana jika karakter setelah penutup adalah kutip tunggal lagi ('), mesin akan bertransisi kembali dari q6 ke q5 untuk melanjutkan pembacaan string tanpa terputus.

e. Mesin mengabaikan teks komentar dan mengubahnya menjadi token COMMENT melalui dua jalur. Untuk komentar kurung kurawal ({...}), mesin masuk ke q7 dan akan terus berputar membaca teks hingga menemukan karakter penutup } di state q8. Sementara untuk komentar kurung bintang ((*...*)), mesin bertransisi ke state penahan di q11 setelah mendeteksi * di dalam komentar. Mesin hanya akan mencapai final state q12 jika karakter selanjutnya adalah). Jika tidak, mesin kembali ke state q10 dengan kondisi except) or * agar tidak berhenti sebelum komentar benar-benar selesai.

f. Untuk simbol statis tunggal yang pasti seperti +, -, *, /, [, dan ;, mesin akan langsung bertransisi dari q0 ke final state masing-masing. Sedangkan untuk simbol relasional dan assignment yang bisa terdiri dari dua karakter, mesin menggunakan state transisi. Contohnya, karakter < membawa mesin ke state q13 (LSS). Jika karakter berikutnya adalah =, mesin berlanjut ke q14 (LEQ), dan jika >, mesin berlanjut ke q15 (NEQ).

2. Implementasi

a. Simulasi Pergerakan DFA (Transisi State) Kelas Lexer menyimpan keseluruhan source code di memori, dan mengandalkan dua fungsi utama untuk menyimulasikan pergerakan di atas pita mesin DFA:

   i. advance(): Fungsi ini berfungsi menggeser pointer pembacaan satu karakter ke depan (transisi state).

src > lexer.cpp > advance()

```cpp
// Maju satu karakter, set null kalau sudah di akhir
void Lexer::advance() {
    current_pos++;

    if (current_pos < source_code.length()) {
        current_char=source_code[current_pos];
    } else {
        current_char = '\0';
    }
}
```

ii. peek(): Fungsi lookahead untuk "mengintip" satu karakter di depan tanpa memajukan pointer. Ini krusial untuk menyelesaikan ambiguitas DFA.

src > lexer.cpp > peek()

```cpp
// Lihat karakter selanjutnya tanpa memajukan posisi (lookahead 1)
char Lexer::peek() {
    size_t peek_pos = current_pos + 1;

    if (peek_pos < source_code.length()) {
        return source_code[peek_pos];
    } else {
        return '\0';
    }
}
```

iii. Fungsi tokenize() bertindak sebagai penggerak State Awal (q0). Menggunakan perulangan while, fungsi ini terus menyeleksi karakter saat ini (current_char) dan mendistribusikannya ke fungsi-fungsi pemindai yang merepresentasikan cabang state berikutnya.

src > lexer.cpp > tokenize()

```cpp
// Fungsi utama tokenisasi: baca seluruh source code dan hasilkan list token
vector<Token> Lexer::tokenize() {
    vector<Token> hasil_token;
    while (current_char != '\0') {
        // Skip whitespace dan komentar dulu sebelum proses
        if (isspace(current_char) || current_char == '{' || (current_char == '(' && peek() == '*')) {
            skipWhitespaceAndComments();
            continue;
        }

        // Karakter huruf -> bisa identifier atau keyword
        if(isalpha(current_char)) {
            hasil_token.push_back(scanIdentOrKeyword());
            continue;
        }

        // Karakter angka -> integer atau real constant
        if (isdigit(current_char)) {
            hasil_token.push_back(scanNumber());
            continue;
        }

        // Karakter petik -> string atau char constant
        if (current_char == '\'') {
            hasil_token.push_back(scanStringOrChar());
            continue;
        }

        // Selain itu masuk ke pemindaian simbol/operator
        hasil_token.push_back(scanSymbol());
    }
    return hasil_token;
}
```

b. Implementasi DFA Dinamis (dfa_dinamis.cpp) Modul ini menangani pembacaan token yang panjangnya tidak terbatas (variable-length) menggunakan perulangan (while). Ini menerapkan prinsip Longest Match, di mana mesin akan terus berputar di state yang sama selama karakter memenuhi syarat.

i. Angka (scanNumber): Membaca digit beruntun. Jika di depan terdapat karakter titik (.) dan angka, ia bertransisi ke state bilangan riil (REALCON), jika tidak, ia berhenti sebagai integer (INTCON).

src > dfa_dinamis.cpp > scanNumber()

```cpp
// Memindai bilangan: integer atau real (jika ada titik desimal)
Token Lexer::scanNumber() {
    string res = "";

    // Baca digit integer dulu
    while (isdigit(current_char)) {
        res += current_char;
        advance();
    }

    // Kalau ketemu titik dan diikuti digit, berarti bilangan real
    if (current_char == '.' && isdigit(peek())) {
        res += current_char;
        advance();

        while (isdigit(current_char)) {
            res += current_char;
            advance();
        }
        return Token(TokenType::REALCON, res);
    }

    return Token(TokenType::INTCON, res);
}
```

ii. Identifier (scanIdentOrKeyword): Terus menerima huruf dan angka secara beruntun (isalnum). Hasil kumpulan karakternya kemudian dikonversi menjadi huruf kecil (lowercase) dan dicocokkan dengan daftar keyword.

src > dfa_dinamis.cpp > scanIdentOrKeyword()

```cpp
// Memindai identifier lalu cek apakah termasuk keyword Pascal
Token Lexer::scanIdentOrKeyword() {
    string res = "";
    res += current_char;
    advance();

    // Terus baca selama masih alfanumerik
    while (current_char != '\0' && isalnum(current_char)) {
        res += current_char;
        advance();
    }

    // Lowercase untuk pencocokan keyword (Pascal case-insensitive)
    string lookup = res;
    transform(lookup.begin(), lookup.end(), lookup.begin(), ::tolower);

    TokenType type = checkKeyword(lookup);
    return Token(type, res);
}
```

iii. String (scanStringOrChar): Jika diawali petik tunggal ('), DFA akan terus menelan (consume) semua karakter. Terdapat logika transisi escape character jika menemukan petik ganda ('').

src > dfa_dinamis.cpp > scanStringOrChar()

```cpp
// Memindai string literal atau char constant yang diapit petik tunggal
Token Lexer::scanStringOrChar() {
    string teks;
    teks += current_char; // Simpan petik pembuka
    advance();

    while (current_char != '\0') {
      if (current_char == '\'') {
        // Petik ganda ('') artinya escape, bukan penutup
        if (peek() == '\'') {
          teks += current_char;
          advance();
          teks += current_char;
          advance();
        } else {
          // Petik penutup
          teks += current_char;
          advance();
          break;
        }
      } else {
        teks += current_char;
        advance();
      }
    }

    // Panjang 3 (misal 'A') berarti char, selain itu string
    if (teks.length() == 3) {
      return Token(TokenType::CHARCON, teks);
    } else {
      return Token(TokenType::STRING, teks);
    }
}
```

iv. Komentar (skipWhitespaceAndComments): Mengabaikan teks hingga menemukan kurung tutup } atau *).

src > dfa_dinamis.cpp > skipWhitespaceAndComments()

```cpp
// Melewati whitespace (spasi, tab, newline) dan komentar Pascal
void Lexer::skipWhitespaceAndComments() {
    while (current_char != '\0') {
        if (isspace(current_char)) {
            advance();
        }
        // Komentar gaya { ... }
        else if (current_char == '{') {
            advance();
            while (current_char != '\0' && current_char != '}') {
                advance();
            }
            if (current_char == '}') advance();
        }
        // Komentar gaya (* ... *)
        else if (current_char == '(' && peek() == '*') {
            advance(); // skip '('
            advance(); // skip '*'
            while (current_char != '\0') {
                if (current_char == '*' && peek() == ')') {
                    advance(); // skip '*'
                    advance(); // skip ')'
                    break;
                }
                advance();
            }
        } else {
            break;
        }
    }
}
```

c. Implementasi DFA Statis (dfa_statis.cpp) Modul ini menangani token pasti yang alur state-nya pendek dan mutlak.

   i. Operator dan Simbol (scanSymbol): Diimplementasikan menggunakan percabangan switch-case. Transisi state digambarkan secara harfiah di sini. Contoh: jika state membaca :, ia masuk ke kasus colon. Jika karakter selanjutnya adalah =, state memanggil advance() dan me-return BECOMES (:=).

src > dfa_statis.cpp > scanSymbol()

```cpp
// Melakukan pemindaian untuk simbol, operator, dan tanda baca
Token Lexer::scanSymbol() {
    switch (current_char) {
        // Operator aritmatika dasar
        case '+':
            advance();
            return Token(TokenType::PLUS, "+");
        case '-':
            advance();
            return Token(TokenType::MINUS, "-");
        case '*':
            advance();
            return Token(TokenType::TIMES, "*");
        case '/':
            advance();
            return Token(TokenType::RDIV, "/");
        // Operator relasional dan logika
        case '=':
            advance();
            if (current_char == '=') {
                advance();
                return Token(TokenType::EQL, "==");
            }
            // Mengembalikan unknown karena di Pascal assignment menggunakan :=
            return Token(TokenType::UNKNOWN, "=");
        case '<':
            advance();
            if (current_char == '=') {
                advance();
                return Token(TokenType::LEQ, "<=");
            }
            if (current_char == '>') {
                advance();
                return Token(TokenType::NEQ, "<>");
            }
            return Token(TokenType::LSS, "<");
        case '>':
            advance();
            if (current_char == '=') {
```

src > dfa_statis.cpp > scanSymbol()

```cpp
            if (current_char == '=') {
                advance();
                return Token(TokenType::GEQ, ">=");
            }
            return Token(TokenType::GTR, ">");
        // Tanda baca dan pengelompokan
        case '(':
            advance();
            return Token(TokenType::LPARENT, "(");
        case ')':
            advance();
            return Token(TokenType::RPARENT, ")");
        case '[':
            advance();
            return Token(TokenType::LBRACK, "[");
        case ']':
            advance();
            return Token(TokenType::RBRACK, "]");
        case ',':
            advance();
            return Token(TokenType::COMMA, ",");
        case ';':
            advance();
            return Token(TokenType::SEMICOLON, ";");
        case '.':
            advance();
            return Token(TokenType::PERIOD, ".");
        case ':':
            advance();
            // Mengecek operator assignment :=
            if (current_char == '=') {
                advance();
                return Token(TokenType::BECOMES, ":=");
            }
            return Token(TokenType::COLON, ":");

        // Menangani karakter yang tidak dikenali
        default: {
            const string unknown_lexeme(1, current_char);
```

src > dfa_statis.cpp > scanSymbol()

```cpp
        // Menangani karakter yang tidak dikenali
        default: {
            const string unknown_lexeme(1, current_char);
            advance();
            return Token(TokenType::UNKNOWN, unknown_lexeme);
        }
    }
}
```

ii. Pencocokan Keyword Instan (checkKeyword): Alih-alih membuat jalur DFA huruf-per-huruf untuk 22 macam keyword Pascal, program mengoptimalkan validasi identifier yang sudah ditangkap dengan menggunakan Hash Map statis (unordered_map). Struktur data ini

memastikan bahwa DFA dapat memastikan apakah sebuah teks itu IDENT biasa atau Keyword (seperti BEGINSY, IFSY) dalam kompleksitas waktu yang sangat cepat dan efisien.

src > dfa_statis.cpp > checkKeyword(const string &)

```cpp
// Memeriksa apakah identifier adalah sebuah keyword
TokenType Lexer::checkKeyword(const string& ident) {
    // Menggunakan static map untuk mapping keyword ke TokenType agar efisien
    static const unordered_map<string, TokenType> keyword_map = {
        // Keyword bawaan Pascal
        {"const", TokenType::CONSTSY},
        {"type", TokenType::TYPESY},
        {"var", TokenType::VARSY},
        {"function", TokenType::FUNCTIONSY},
        {"procedure", TokenType::PROCEDURESY},
        {"array", TokenType::ARRAYSY},
        {"record", TokenType::RECORDSY},
        {"program", TokenType::PROGRAMSY},
        {"begin", TokenType::BEGINSY},
        {"if", TokenType::IFSY},
        {"case", TokenType::CASESY},
        {"repeat", TokenType::REPEATSY},
        {"while", TokenType::WHILESY},
        {"for", TokenType::FORSY},
        {"end", TokenType::ENDSY},
        {"else", TokenType::ELSESY},
        {"until", TokenType::UNTILSY},
        {"of", TokenType::OFSY},
        {"do", TokenType::DOSY},
        {"to", TokenType::TOSY},
        {"downto", TokenType::DOWNTOSY},
        {"then", TokenType::THENSY},

        // Operator berbentuk kata
        {"not", TokenType::NOTSY},
        {"and", TokenType::ANDSY},
        {"or", TokenType::ORSY},
        {"div", TokenType::IDIV},
        {"mod", TokenType::IMOD},
    };
```

src > dfa_statis.cpp > checkKeyword(const string &)

```cpp
TokenType Lexer::checkKeyword(const string& ident) {

    // Mengubah identifier ke lowercase karena Pascal bersifat case-insensitive
    string lower_ident;
    lower_ident.reserve(ident.size());

    for (const char ch : ident) {
        lower_ident += static_cast<char>(
            tolower(static_cast<unsigned char>(ch)));
    }

    // Cek apakah identifier ada di dalam map keyword
    const auto it = keyword_map.find(lower_ident);
    if (it != keyword_map.end()) {
        return it->second;
    }
    return TokenType::IDENT; // Jika tidak ditemukan, kembalikan sebagai identifier biasa
}
```

C. PENGUJIAN

No	Input	Output

1	test > milestone-1 > input-1.txt
	program LuasLingkaran;
	
	const
	  PI = 3.14159;
	
	var
	  jarijari, luas: real;
	
	begin
	  jarijari := 7.5;
	  luas := PI * jarijari * jarijari;
	  writeln('Luas = ', luas);
	end.
		test > milestone-1 > output-1.txt
		programsy
		ident (LuasLingkaran)
		semicolon
		constsy
		ident (PI)
		UNKNOWN (=)
		realcon (3.14159)
		semicolon
		varsy
		ident (jarijari)
		comma
		ident (luas)
		colon
		ident (real)
		semicolon
		beginsy
		ident (jarijari)
		becomes
		realcon (7.5)
		semicolon
		ident (luas)
		becomes
		ident (PI)
		times
		ident (jarijari)
		times
		ident (jarijari)
		semicolon
		ident (writeln)
		lparent
		string ('Luas = ')
		comma
		ident (luas)
		rparent
		semicolon
		endsy
		period

2	test > milestone-1 > input-2.txt
	`program GanjilGenap;`
	
	`var`
	`  angka: integer;`
	
	`begin`
	`  angka := 15;`
	`  if (angka mod 2 == 0) then`
	`  writeln('Genap')`
	`  else`
	`  writeln('Ganjil');`
	`end.`
		test > milestone-1 > output-2.txt
		`programsy`
		`ident (GanjilGenap)`
		`semicolon`
		`varsy`
		`ident (angka)`
		`colon`
		`ident (integer)`
		`semicolon`
		`beginsy`
		`ident (angka)`
		`becomes`
		`intcon (15)`
		`semicolon`
		`ifsy`
		`lparent`
		`ident (angka)`
		`imod`
		`intcon (2)`
		`eql`
		`intcon (0)`
		`rparent`
		`thensy`
		`ident (writeln)`
		`lparent`
		`string ('Genap')`
		`rparent`
		`elsesy`
		`ident (writeln)`
		`lparent`
		`string ('Ganjil')`
		`rparent`
		`semicolon`
		`endsy`
		`period`

3	test > milestone-1 > input-3.txt
	`program KonversiSuhu;`
	
	`var`
	`  celsius, fahrenheit: real;`
	
	`begin`
	`  (* Inisialisasi suhu di bawah nol *)`
	`  celsius := -10.5;`
	
	`  { Rumus F = C * 9/5 + 32 }`
	`  fahrenheit := celsius * 9 div 5 + 32;`
	
	`  writeln('{Hasil} Suhu F = ', fahrenheit);`
	`end.`
		test > milestone-1 > output-3.txt
		`programsy`
		`ident (KonversiSuhu)`
		`semicolon`
		`varsy`
		`ident (celsius)`
		`comma`
		`ident (fahrenheit)`
		`colon`
		`ident (real)`
		`semicolon`
		`beginsy`
		`ident (celsius)`
		`becomes`
		`minus`
		`realcon (10.5)`
		`semicolon`
		`ident (fahrenheit)`
		`becomes`
		`ident (celsius)`
		`times`
		`intcon (9)`
		`idiv`
		`intcon (5)`
		`plus`
		`intcon (32)`
		`semicolon`
		`ident (writeln)`
		`lparent`
		`string ('{Hasil} Suhu F = ')`
		`comma`
		`ident (fahrenheit)`
		`rparent`
		`semicolon`
		`endsy`
		`period`

4

test > milestone-1 > input-4.txt

```
program HitungExposure;

var
  iso, shutter: integer;
  aperture: real;

begin
  { Simulasi Segitiga Exposure }
  iso := 400;
  shutter := 250;
  aperture := 2.8;

  if (iso >= 800) and (aperture <= 1.4) then
    writeln('Foto overexposed');
end.
```

test > milestone-1 > output-4.txt

```
programsy
ident (HitungExposure)
semicolon
varsy
ident (iso)
comma
ident (shutter)
colon
ident (integer)
semicolon
ident (aperture)
colon
ident (real)
semicolon
beginsy
ident (iso)
becomes
intcon (400)
semicolon
ident (shutter)
becomes
intcon (250)
semicolon
ident (aperture)
becomes
realcon (2.8)
semicolon
ifsy
lparent
ident (iso)
geq
intcon (800)
rparent
andsy
lparent
ident (aperture)
leq
realcon (1.4)
rparent
```

```
rparent
thensy
ident (writeln)
lparent
string ('Foto overexposed')
rparent
semicolon
endsy
period
```

5	test > milestone-1 > input-5.txt
	program HitungFaktorial;
	
	var
	n, hasil: integer;
	
	begin
	n := 5;
	hasil := 1;
	repeat
	hasil := hasil * n;
	n := n-1;
	until(n == 0);
	writeln('Hasil = ', hasil);
	end.
		test > milestone-1 > output-5.txt
		programsy
		ident (HitungFaktorial)
		semicolon
		varsy
		ident (n)
		comma
		ident (hasil)
		colon
		ident (integer)
		semicolon
		beginsy
		ident (n)
		becomes
		intcon (5)
		semicolon
		ident (hasil)
		becomes
		intcon (1)
		semicolon
		repeatsy
		ident (hasil)
		becomes
		ident (hasil)
		times
		ident (n)
		semicolon
		ident (n)
		becomes
		ident (n)
		minus
		intcon (1)
		semicolon
		untilsy
		lparent
		ident (n)
		eql
		intcon (0)
		rparent
		semicolon
		semicolon
		ident (writeln)
		lparent
		string ('Hasil = ')
		comma
		ident (hasil)
		rparent
		semicolon
		endsy
		period

D. KESIMPULAN DAN SARAN

1. Kesimpulan

Secara keseluruhan, *Lexical Analyzer* Arion berhasil diimplementasikan secara modular menggunakan prinsip DFA murni. Program terbukti efisien dalam kinerjanya, tangguh dalam menangani berbagai jebakan karakter (*edge cases*), dan mampu menghasilkan *output* token yang 100% sesuai dengan spesifikasi format yang diminta.

2. Saran

Untuk pengembangan selanjutnya, kami menyarankan penambahan fitur pelacakan baris dan kolom pada token untuk mempermudah *Syntax Analysis*, peningkatan sistem penanganan *error* agar lebih spesifik dan interaktif, serta senantiasa mempertahankan struktur *Clean Code* demi kelancaran integrasi pada tahapan *milestone* berikutnya.

E. LAMPIRAN

Github: https://github.com/Rmadhian/DGN-Tubes-IF2224-2026
Workspace Diagram:
https://drive.google.com/file/d/1ePa5FFHOAYrkhZwaiDOImksFZQCQqEVa/view?usp=sharing
Tabel Kontribusi:

NIM	Kontribusi	Persentase
13523150	Mengerjakan dfa_statis.cpp.	30%
13524117	Mengirim foto selfie.	0.1%
13524120	Mengerjakan dfa_dinamis.cpp, membantu main.cpp, mengerjakan laporan.	34.9%
13524126	Membuat lexer.h, mengerjakan lexer.cpp, main.cpp, Makefile, membuat diagram, dan laporan.	35%